Future Generation Computer Systems 174 (2026) 107937

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

Efficient complex task offloading in blockchain-secured edge computing using a many-objective evolutionary algorithm

Shilong Li [a,b], Wenping Ma [a,*]

[a] School of Telecommunications Engineering, Xidian University, Xi'an 710071, China
[b] State Key Laboratory of Intelligent Vehicle Safety Technology, Chongqing 401133, China

ARTICLE INFO

Keywords:
Complex workflow applications
Edge computing
Secure task offloading
Reputation model
Fast blockchain consensus
Many-objective evolutionary

ABSTRACT

With the rapid growth of mobile services and complex workflow applications, Mobile edge computing (MEC) has become a promising paradigm for enhancing user experience. However, the uneven distribution of computing resources, the complexity and diversity of emerging workflow applications, and the risk of privacy leakage during task offloading pose significant challenges to resource allocation efficiency and security. To this end, we propose the BICRA framework, which innovatively integrates the EigenTrust-TW reputation model, which adapts to dynamically changing MEC scenarios, and the RBFT-SMaRT fast blockchain consensus protocol, which employs a two-phase modular architecture and supports fast Byzantine fault tolerance to ensure secure task offloading and optimize the performance of latency-sensitive applications. Additionally, BICRA adopts MaOEA-FT, a novel many-objective evolutionary algorithm that comprehensively considers execution time, energy consumption, load balancing, and task rewards for complex workflow applications, aiming to adapt to data-intensive applications and more closely resembling actual MEC scenarios. Compared with existing MEC and blockchain solutions, the BICRA framework innovatively achieves the synergy of many-objective optimization and blockchain performance optimization, effectively addressing the dual challenges of efficient resource allocation and secure task offloading. Security analysis demonstrates that the BICRA architecture provides a secure and reliable resource allocation platform. Experimental results show that BICRA significantly outperforms existing schemes in key performance indicators: transaction latency is reduced by 74.85%, system throughput is improved by 60.75%, many-objective evolution convergence speed is increased by nearly 60%, and the resource allocation scheme reaches the optimal.

1. Introduction

Rapid advances in the Internet of Things (IoT) and mobile communications technology have given rise to a range of data-intensive and latency-sensitive applications, such as smart healthcare [1], autonomous vehicles [2], and virtual reality [3]. These cutting-edge applications have been deeply integrated into daily life. Although the computing power of today's mobile user devices (MUDs) has been greatly improved, it is still difficult to handle these complex workflow applications independently due to the inherent limitations of hardware resources. Mobile Edge Computing (MEC) has emerged as an effective solution to the above problems [4]. Unlike cloud computing, MEC deploys edge server nodes (ENs) closer to the MUDs, significantly reducing data transmission latency while alleviating congestion in the core network [5]. With limited computing resources in MUDs, offloading tasks to ENs can reduce task execution time of tasks and save resources. Therefore, task offloading has become a popular research topic in edge computing [6].

Chen et al. [7] proposed an actor–critic mechanism with two embeddings for DAGs-based multiple dependent tasks computation offloading to get the optimal computation offloading strategy. Li et al. [8], from the joint optimization perspective, formulated the problem of computational resource allocation and task offloading as a mixed-integer nonlinear programming (MINP) problem and introduced a two-stage heuristic optimization algorithm based on a genetic algorithm. However, both [7,8] overlooked the impact of load balancing and task rewards on task offloading strategies. Another research dimension focuses on many-objective computing offloading problems, optimizing execution time and energy consumption for mobile device applications. A differentiated and decoupled multi-objective evolutionary algorithm (MOEA/D) balances these two objectives and effectively addresses this technical challenge [9]. However, this study does not consider the optimization of energy consumption and load balancing. Cui et al. [10] designed a many-objective optimized edge and end

* Corresponding author.
E-mail addresses: sl_li@stu.xidian.edu.cn (S. Li), wp_ma@mail.xidian.edu.cn (W. Ma).

https://doi.org/10.1016/j.future.2025.107937
Received 20 July 2024; Received in revised form 7 January 2025; Accepted 25 May 2025
Available online 11 June 2025
0167-739X/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1
Summary of extant studies.

Scheme	Blockchain			Optimization objectives				Complex workflow
	Security	Latency	Throughput	Execution time	Energy consumption	Load balancing	Reward	
[7]	✗	✗	✗	✓	✓	✗	✗	✗
[9]	✗	✗	✗	✓	✗	✗	✓	✗
[11]	✗	✗	✗	✓	✓	✓	✗	✓
[12]	✓	✓	✓	✗	✓	✗	✗	✗
[13]	✓	✗	✓	✗	✓	✗	✗	✗
[8,10]	✗	✗	✗	✓	✓	✓	✗	✗
[14,15]	✓	✗	✗	✓	✗	✗	✗	✗
[16,17]	✓	✓	✗	✗	✓	✗	✓	✗
Ours	✓	✓	✓	✓	✓	✓	✓	✓

Note: "✓" denotes deal with a limitation; "✗" denotes does not deal with a limitation.

collaborative computation offloading (MaOEECCO) model and developed a many-objective evolutionary algorithm with load constraints; Peng et al. [11] framed computing offload and resource allocation as a multi-objective optimization problem for the industrial internet scenario and proposed an intelligent optimization method for end–edge–cloud collaboration. Nevertheless, neither [10,11] incorporated task gains into the optimization objectives for computation offloading and resource allocation.

There are two significant limitations in the existing research [7–11]: (1) it fails to comprehensively consider the synergistic impact of many-dimensional factors such as execution time, energy consumption, load balancing and task reward, on computing resource allocation and task offloading; (2) the existing workflow models are insufficient to adequately characterize the diversity and complexity of real-world application scenarios. To address these limitations, we first formalize the computing resource allocation and task offloading problem into the MaOEECCO model, which simultaneously considers four key optimization objectives (execution time, energy consumption, load balancing, and task rewards) and five constraints, aiming to derive a more practical and efficient resource allocation scheme. Second, we innovatively propose a new complex workflow model that integrates dependent and independent workflows, enabling a more accurate representation of the complex characteristics of real workflow applications.

In addition, the above studies also overlook the security issues arising from the heterogeneity of computational resources among ENs and the privacy leakage due to task offloading. Blockchain is widely regarded as a promising data security technology [18]. The characteristics of blockchain, such as decentralization, sharing, and tamper resistance, can be used to ensure the integrity and security of data. Furthermore, due to the similar decentralization characteristics of blockchain and MEC, combining the two advanced technologies will become very natural [14]. Recently, various models and frameworks incorporating blockchain technology have been proposed, including the secure task offloading framework introduced in [16], which ensures security while achieving an optimized balance of execution time and energy consumption. Cui et al. [12] proposed an edge computing architecture supporting sharding and proposed a many-objective evolutionary algorithm based on information entropy to derive the optimal sharding scheme. Ye et al. [13] proposed a collaborative optimization framework for blockchain-enabled MEC systems, employing a deep reinforcement learning approach based on Asynchronous Dominant Actor Criticism (A3C) to optimize energy consumption and transaction throughput. In addition, Wang et al. [17] proposed an incentive-aware blockchain-assisted smart edge caching and computation offloading scheme for IoT, offering a solution for minimizing task completion costs.

Although current research [12,13,16,17] has explored on the task offloading mechanism, a significant research gap remains in optimizing blockchain performance for MEC systems. Blockchain performance optimization research primarily focuses on consensus protocols, aiming to address two core performance metrics: transaction latency and throughput [15,19,20]. However, mainstream consensus mechanisms exhibit notable limitations in MEC scenarios: Proof-of-work (PoW) [21] is known for its time-consuming consensus process and is not suitable for MEC systems; Proof-of-stake (PoS) [20] is vulnerable to attacks as the highest interest miner node chosen for block generation is an easy target for attackers; and Byzantine fault tolerant consensus (PBFT) [19] consensus still has high computational complexity and latency since it all needs to verify the blocks generated by all nodes. Therefore, given the need for low latency and high performance in MEC systems, there is a need to design a novel consensus protocol that significantly reduces transaction latency and improves throughput without compromising the level of security.

In summary, existing frameworks exhibit significant shortcomings in terms of security and comprehensive optimization objectives: some schemes are vulnerable to various attack threats, while others fail to adequately address the many-objective optimization requirements for computational resource allocation and task offloading. As shown in Table 1, existing studies demonstrate notable limitations in both computational efficiency and secure offloading. To address these issues, we propose a blockchain-integrated many-objective optimized edge-end collaborative computational resource allocation (BICRA) framework, which incorporates the following innovative features: (1) achieving joint optimization of computing resource allocation and task offloading; (2) maximizing computational efficiency and transaction throughput under security-level constraints; and (3) effectively addressing the challenges of secure offloading in MEC scenarios. The contributions of this paper are as follows:

1. We design a reputation-based BFT-SMaRt consensus protocol (RBFT-SMaRt) to record and manage the task offloading process, which significantly reduces transaction confirmation latency, improves data throughput, ensures fair execution of transactions, and fits the needs of high-performance, low-latency MEC systems;
2. We deploy a smart contract that performs reputation computation using a three-weighted EigenTrust algorithm (EigenTrust-TW). This identifies and selects reliable nodes to participate in the consensus process and is capable of continuously updating node reputations in a dynamically changing evaluation environment, thus facilitating a more robust and efficient resource allocation;
3. A more realistic model of complex workflow tasks is proposed, which includes both independent and dependent workflows. In addition, the resource allocation and task offloading problem is modeled and formulated as a many-objective optimization problem, with the objective of achieving optimal resource utilization and task handling strategies in an edge-end collaborative environment;
4. A fast converging many-objective evolutionary algorithm based on fitness tracking, MaOEA-FT, is developed to comprehensively optimize the four key objectives: execution time, energy consumption, load balancing, and task rewards. This is done in order to generate offloading decisions that are more closely aligned with the actual needs of MEC, and to effectively improve the efficiency and practicality of resource allocation.

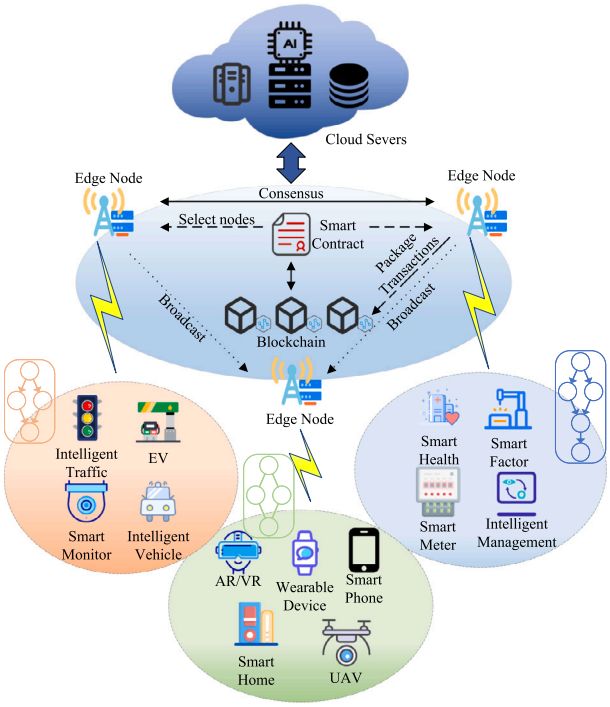


Fig. 1. A blockchain-integrated many-objective optimized edge-end collaborative computational resource allocation framework (BICRA).

The structure of this article is as follows. In Section 2, we detail how blockchain empowers MEC, highlighting BICRA framework and proposing a fast consensus protocol. Section 3 delves into the use of smart contracts for reputation computing. In Section 4, we introduced an innovative many-objective evolutionary algorithm. Section 5 analyzes BICRA framework's security and presents the experimental results. Finally, Section 6 summarizes the entire content and summarizes the core contributions of the article.

2. BICRA key approaches and designs

In this section, we present an edge-end collaborative computing resource allocation framework using blockchain and smart contract.

2.1. Structure overview of BICRA

Using blockchain technology, interactions between mobile devices can be securely recorded and verified, thus enhancing the protection of data in transmission. In order to maintain the reliability and integrity of transmitted data during resource allocation and task offloading, BICRA is designed as shown in Fig. 1. Each EN is equipped with an edge server, and the set of servers is denoted as $\mathbb{Z} = \{z_1, z_2, z_3, \ldots, z_m, \ldots, z_M\}$, where M is the number of ENs. The set of MUDs is denoted as $\mathbb{U} = \{u_1, u_2, u_3, \ldots, u_n, \ldots, u_N\}$, where N is the number of MUDs. Each EN dynamically builds a blockchain network using intelligent agents, where each EN can act as a respective blockchain network node. The number of blockchain nodes corresponds to the number of ENs in the actual scenarios. Each MUD has complex workflow applications that need to be offloaded for execution. Each EN collects immutable offload processes, packages them into transactions, and submits them to the blockchain for verification and consensus at each time slot T. Once verified and consented to by other ENs, the offload transactions are recorded in block format and added to the blockchain.

Furthermore, illegal and malicious ENs pose threats to data security and privacy. To improve the efficiency of the blockchain consensus mechanism requires strict verification and authentication of EN identities. To this end, we propose implementing Reputation Computing Smart Contracts (RCSC) within the BICRA framework. Reliable ENs are selected to participate in the consensus through RCSC to improve the consensus efficiency while ensuring security management. Meanwhile, the AI-enhanced cloud server deploys the MaOEA-FT algorithm that intelligently manages computational resource allocation and provides recommendations for task offloading for MUDs.

Protocol 1 Secure Information Sharing Protocol of BICRA

1. The EN_m broadcasts its service capabilities to MUD_n within its coverage area:
   $EN_m \rightarrow MUD_n : E_{K^{pu}_{Pseu^s_n}}(timestamp \| Data_{EN_m} \| Cert_{EN_m} \| Sign_{EN_m}(Data_{EN_m}))$,
   where $Data_{EN_m} = E_{K^{pr}_{EN_m}}(timestamp \| resource_m \| location_m \| price_m \| t_m)$.
2. The MUD_n utilizes the MaOEA-FT algorithm to make task offloading decisions and select the task offloading target EN_m:
   $MUD_n \rightarrow EN_m : E_{K^{pu}_{EN_m}}(timestamp \| Req_{Pseu^s_n} \| Cert_{Pseu^s_n} \| Sign_{Pseu^s_n}(Req_{Pseu^s_n}))$,
   where $Req_{Pseu^s_n} = E_{K^{pr}_{Pseu^s_n}}(timestamp \| code_n \| data_n)$.
3. After completing the task, the MUD_n will upload the evaluation of this service to assist the EigenTrust-TW algorithm in updating the reputation of EN_m:
   $MUD_n \rightarrow EN_m : E_{K^{pu}_{EN_m}}(timestamp \| Eva_{Pseu^s_n} \| Cert_{Pseu^s_n} \| Sign_{Pseu^s_n}(Eva_{Pseu^s_n}))$,
   where $Eva_{Pseu^s_n} = E_{K^{pr}_{Pseu^s_n}}(timestamp \| s'_{nm})$.
4. The EN_m will package the uninstallation process into blocks. Following the completion of the RBFT-SMaRt consensus protocol, EN_m broadcast the block to all ENs for storage:
   $EN_m \rightarrow All : Data_block = (timestamp \| Data_n \| Cert_{EN_m} \| Sign_{EN_m}(Data_n))$,
   where
   $Data_n = E_{K^{pr}_{EN_m}}(timestamp \| mode_n \| code_n \| data_n)$.

2.2. Secure information interaction protocol of BICRA

As previously discussed, within the BICRA framework, each EN broadcasts its service capability to MUDs within its coverage area. Upon receiving this message, MUD_n make an offloading decision using the MaOEA-FT algorithm, taking into account the EN's service capabilities and evaluating the service process. Subsequently, the offloading transaction is recorded in a block and appended to the current blockchain after verification and consent from other ENs.

To ensure the security of information interaction within the BICRA system, we have designed a secure information interaction protocol detailed in Protocol 1. We illustrate the use of EN_m and MUD_n as examples. Here $Pseu^s_n$ represents the current pseudonym of MUD_n, with $K^{pu}_{Pseu^s_n}$ and $K^{pr}_{Pseu^s_n}$ denoting its public and private keys, respectively. Similarly, $K^{pu}_{EN_m}$ and $K^{pr}_{EN_m}$ refer to the public and private keys of EN_m. $Cert_{EN_m}$ and $Sign_{EN_m}$ correspond to the certificate and signature of EN_m, while $Cert_{Pseu^s_n}$ and $Sign_{Pseu^s_n}$ represent those of MUD_n. $Data_{EN_m}$ is encrypted using $K^{pr}_{EN_m}$, and includes computing resources $resource_m$, geographic location $location_m$, initial price $price_m$ and global reputation t_m. $Req_{Pseu^s_n}$ is encrypted with $K^{pr}_{Pseu^s_n}$ and includes the code block $code_n$ and input data $data_n$. $Eva_{Pseu^s_n}$ denotes the local reputation value s'_{nm} encrypted by $K^{pr}_{Pseu^s_n}$. $Data_n$ is encrypted using $K^{pr}_{EN_m}$, includes $code_n$, $data_n$, and task mode $mode_n$.

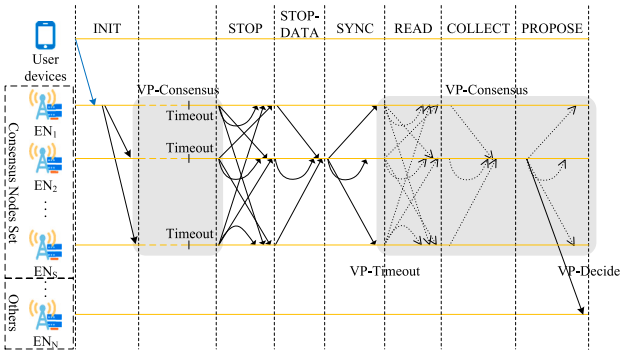


Fig. 2. Synchronization phase of RBFT-SMaRt consensus protocol.

2.3. RBFT-SMaRt consensus protocol

Given the dynamic nature of the MEC system, minimizing blockchain consensus latency is crucial for timely updating of user computational offload information. Compared to these algorithms [19–21], BFT-SMaRt [22] is widely favored as it exhibits lower latency and higher throughput, while enhancing the degree of modularity to be more in line with actual deployment requirements. We propose the RBFT-SMaRt consensus protocol with two phases that combines reputation computing with BFT-SMaRt, tailored for MEC system. Specifically, the RBFT-SMaRt consensus protocol first needs to go through the steps of SELECT and BROADCAST, and then enters the normal or synchronization phase.

- **SELECT**: In the BICRA framework, the S nodes with the highest reputation values are selected based on their reputation values to form the consensus node set participating in the RBFT-SMaRt consensus protocol, denoted as $\mathbf{EN}^{con}$. The maximum number of malicious nodes to be tolerated is f, where $f = (S-1)/3$. The selection of $\mathbf{EN}^{con}$ will be described in detail in Section 3. The remaining nodes will be used as candidate node sets, denoted as $\mathbf{EN}^{can}$. In each round of reaching consensus, a node will be selected from $\mathbf{EN}^{con}$ as the speaker, denoted as EN_x. EN_x selected by $x = (h \bmod S) + 1$, where h refers to the current block height. EN_x organizes the recently generated offloading process records into a block and adds the block to the current chain through the consensus protocol among $\mathbf{EN}^{con}$. $\mathbf{EN}^{can}$ can obtain the latest blocks by accessing the system, but does not participate in the consensus process.
- **BROADCAST**: The EN responsible for allocating computing resources for the offloading task broadcasts the offloading process record $Data_block$ to the nodes in the $\mathbf{EN}^{con}$ within the blockchain's consensus node network.

In the normal phase, RBFT-SMaRt uses the VP-Consensus protocol to execute a correct sequence of operations in a consensus instance. If the consensus fails, the synchronization phase is initiated and the VP-Consensus is run synchronously. As shown in Fig. 2, each specific step of RBFT-SMaRt can be clearly observed, the blue line represents the task information of MUD offloading, and the dashed portion corresponds to the messages of the VP-consensus protocol.

- **INIT**: The process comprises two steps, SELECT and BROADCAST. For the purposes of this discussion, we shall assume that the initial speaker is EN_1. In the event of an attack on EN_1 or in the case of more than f timeouts for the same request, which indicates a failure of the consensus, the speaker is replaced.
- **STOP**: At that moment all local countdown timers are suspended and then the synchronization phase is initiated, the speaker is replaced by default with the next node (i.e. EN_2), and a STOP message is broadcast. A node counts the number of nodes that have received the STOP message locally to reach the number of acknowledgments (i.e., at least $2f + 1$). Once this number has been determined, the local node then determines that the speaker replacement is successful and sends a STOP-DATA message to the new speaker.
- **STOP-DATA**: The EN_2 receives the STOP-DATA message up to the number of acknowledgments and sends a SYNC message to the other nodes, containing the original message decision.
- **SYNC**: When the node receives the SYNC message, it performs the same calculations as the original message, thus verifying that the speaker's information is correct. The READ message is then broadcast outward.
- **READ**: The node locally collects READ messages up to the number of acknowledgments, then sends a COLLECT message to the speaker.
- **COLLECT**: The EN_2 receives a COLLECT message up to the number of acknowledgments, then sends a PROPOSE message representing that this original message state has been synchronized.
- **PROPOSE**: All nodes, including those not participating in the consensus, receive the PROPOSE message from the speaker and complete the local processing of the message.

If the original speaker does not perform an exception, there is no requirement for replacement. The consensus processing of the message can be completed in three steps: READ-COLLECT-PROPOSE. In other words, the normal phase of RBFT-SMaRt reduces the number of message interactions and helps to reduce the transaction latency.

3. A smart contract based on EigenTrust-TW reputation model

This section extends the EigenTrust reputation model [23] by incorporating node characteristics such as event timeliness, interaction frequency, and geographic location similarity.

3.1. Basic concepts and symbols

The binary tuple $(sat(i,j), unsat(i,j))$ represents the evaluation of node i regarding node j, where $i \neq j$. Here, $sat(i,j)$ and $unsat(i,j)$ denote the number of satisfactory and unsatisfactory evaluations, respectively, accumulated by node i regarding node j in historical transactions. These values are maintained as local historical records.

After two nodes complete an interaction, both parties evaluate the interaction, and the evaluation results include both positive and negative. In EigenTrust reputation model, the local reputation values are represented as

$$s_{ij} = sat(i,j) - unsat(i,j). \tag{1}$$

The updated local reputation value after standardization is as

$$c_{ij} = \frac{\max(s_{ij}, 0)}{\sum_{k=1}^{M} \max(s_{ik}, 0)}. \tag{2}$$

The global reputation value of any node j is represented as

$$t_j^{k+1} = c_{1j} t_1^k + c_{2j} t_2^k + \cdots + c_{nj} t_n^k, \tag{3}$$

where k denotes the number of iterations, n denotes the number of nodes associated with node j, t_j^{k+1} and t_j^k represent the current global reputation value and past reputation value of node j.

3.2. The influencing factors of node characteristics

3.2.1. Event timeliness

In this paper, the calculation of local reputation values incorporates the influence of time factors on reputation values. To represent the time

decay factor γ, a sigmoid function is introduced.

$$\gamma = \frac{1}{1 + e^{\theta(t_{re} - t_{tr})}}, \tag{4}$$

where t_{tr} denotes the time of interaction, and t_{re} represents the current time. The degree of decay increases as the interaction time deviates from the current time, and vice versa. The parameter θ serves is a scaling factor for the time decay factor to adjust the weighting of event timeliness.

3.2.2. Interaction frequency

It is widely recognized that a higher frequency of interaction indicates that node i has a more comprehensive knowledge of node j based on previous interactions. This increased familiarity enables more accurate and reliable reputation assessment. Interaction frequency is quantified as the ratio of number of interactions between nodes i and j relative to the average number of interactions node i interacts with other nodes over a specified time interval T. It is mathematically represented as

$$IF_{ij} = \frac{N_{ij}}{\overline{N_i}}, \tag{5}$$

where $N_{ij} = sat(i,j) + unsat(i,j)$, $\overline{N_i} = \frac{1}{|M|}\sum_{j=1}^{M} N_{ij}$. M is the total number of nodes that interact with node i within a time window T. The higher the frequency of interaction, the higher the reputation weight.

3.2.3. Geographic location information similarity

Nodes in close geographical proximity are presumed to have easier communication and cooperation, warranting higher trust weight allocation, and vice versa [24]. Geographic location information typically comprises two elements: longitude lo and latitude la, which can be acquired through satellite-based radio navigation systems (e.g. GPS) or cellular towers. Therefore, the format of geographic location information usually follows the pattern: $(lo, la, timestamp)$. The task offloading conducted by mobile user devices is inherently tied to their locations and exhibits a spatial range. To address this, we introduce a distance decay factor. As the distance between two nodes increases, their reputation evaluation weights should decay accordingly.

Node i obtains the latest geographic location information $(lo_j, la_j, timestamp_j)$ of node j. Node i first needs to determine whether the $timestamp_j$ of the geographic information is up-to-date. Then use the Haversine formula to calculate the Euclidean distance between the two, denoted as

$$d_{ij} = 2r \arcsin\sqrt{\sin^2\left(\frac{la_j - la_i}{2}\right) + \cos(la_i)\cos(la_j)\sin^2\left(\frac{lo_j - lo_i}{2}\right)}, \tag{6}$$

where $r = 6371.393$ km (the average radius of the Earth), lo_i, la_i is the longitude and latitude of node i, respectively, lo_j, la_j is the longitude and latitude of node j. Then use exponential decay to map the distance d_{ij} to

$$GE_{ij} = e^{-\lambda d_{ij}}, \tag{7}$$

where λ is a parameter in the exponential distribution used to adjust the weight ratio.

3.3. EigenTrust-TW algorithm

Based on the above node characteristics, we propose the EigenTrust-TW reputation model for MEC scenarios. Definition $positive_{ij}^{t_{tr}}$ is the positive evaluation of node i towards node j at time t_{tr}, $negative_{ij}^{t_{tr}}$ is the negative evaluation of node j by node i at time t_{tr}. $positive_{ij}^{t_{tr}}$ and $negative_{ij}^{t_{tr}}$ are all uniformly quantified as a value of 1. Substitute Eqs. (4) and (7) into the binary tuple, introduce weighting factors for event timeliness and geographic information similarity. Thus, $sat'(i,j)$ and $unsat'(i,j)$ replace the $sat(i,j)$ and $unsat(i,j)$ in the local trust values separately, denoted as

$$sat'(i,j) = \alpha_1 \cdot \gamma \cdot GE_{ij} \sum_{t=t_{tr}}^{t_{re}} positive_{ij}^{t}, \tag{8}$$

$$unsat'(i,j) = \beta_1 \cdot \gamma \cdot GE_{ij} \sum_{t=t_{tr}}^{t_{re}} negative_{ij}^{t}, \tag{9}$$

where α_1 and β_1 represent the weights of positive and negative evaluations, respectively. Moreover, the weighting of negative evaluations is higher than that of positive evaluations $\alpha_1 + \beta_1 = 1$, and $\alpha_1 < \beta_1$. Substitute Eq. (5) into Eq. (1), introduce interaction frequency information, and update the local reputation value as

$$s'_{ij} = (sat'(i,j) - unsat'(i,j)) \cdot IF_{ij}. \tag{10}$$

The Eq. (2) is rewritten as

$$c'_{ij} = \frac{\max(s'_{ij}, 0)}{\sum_{k=1}^{M} \max(s'_{ik}, 0)}. \tag{11}$$

Establishing trust is crucial, for early nodes that have recently joined the network or are inactive. These nodes that are trusted at the beginning of joining the network are called pre-trusted nodes and form a P set. This distinction is particularly important for that lack pre-established trust relationships. Consequently, the local trust value is redefined as

$$c'_{ij} = \begin{cases} \frac{\max(s'_{ij}, 0)}{\sum_{k=1}^{M} \max(s'_{ik}, 0)}, & \text{if } \sum_{k=1}^{M} \max(s'_{ik}, 0) \neq 0 \\ p_j, & \text{otherwise,} \end{cases} \tag{12}$$

where

$$p_j = \begin{cases} 1/|P|, & \text{if } j \in P \\ 0, & \text{otherwise,} \end{cases} \tag{13}$$

where $|P|$ is the number of pre-trusted nodes.

At times, malicious users within the BICRA framework collaborate to form a malicious collective. Within this collective, they assign artificially high reputation values to each other while attributing low reputation values to honest nodes. To address this challenge, (12) is substituted into (3) to recalibrate the global reputation value of node i. This process results in the rewriting of (3) as

$$t_j^{k+1} = (1-a)(c'_{1j}t_1^k + c'_{2j}t_2^k + \cdots + c'_{nj}t_n^k) + a \cdot p_i, \tag{14}$$

where a denotes the weight of the pre-trusted nodes in the global reputation evaluation, and $a \in [0,1]$.

3.4. Selecting consensus node sets for RBFT-SMaRt

The MUD_n selects the task offloading object EN_m through the computing resource allocation scheme detailed in Section 4. Then MUD_n selects the $\mathbf{EN}^{con}$ by comparing the global reputation values of all ENs. For a period of time, the $\mathbf{EN}^{con}$ chosen by MUD_n includes EN_m. This is because the EigenTrust-TW reputation model takes into account geographic location information, which is closely related to execution time and energy consumption. Therefore when MUD_n chooses to offload the task object, the reputation value of EN_m is relatively high for MUD_n and EN_m belongs to the $\mathbf{EN}^{con}$. The node set in descending order is denoted as

$$\mathbf{SET}^{EN} = Sort_{j \in [1:M]}(t_j), \tag{15}$$

where t_j represents the global reputation of the EN_j, the $Sort$ function is used to sort EN_j in descending order. The specific steps to find the $\mathbf{EN}^{con}$ are as follows in Algorithm 1.

The high computational overhead of PoW and the centralization and vulnerability issues of PoS make them unsuitable for MEC environments. Conversely, the RBFT-SMaRt consensus mechanism, leveraging

Algorithm 1 Selecting consensus node sets using RCSC

Input: The local opinions c'_{xn} and c'_{xi}, the global reputation $t^k_{EN_x}$ and $t^k_{MUD_x}$.
Output: The consensus node set $\mathbf{EN}^{con}$.

```
for n = 1 to N do
    MUD_n download the latest block information;
    for EN_x that has interacted with MUD_n do
        Obtain c'_xn and t^k_{EN_x} from the latest block information;
    end for
    Trigger RCSC on MUD, t^{k+1}_{MUD_n} be obtained as (14);
    Determine EN_m through algorithm MaOEA-FT;
    Exchange certificates and signatures with EN_m;
    for i ∈ M do
        Trigger RCSC on EN;
        for MUD_x that has interacted with EN_i do
            Obtain c'_xi and t^k_{MUD_x} from the latest block information;
        end for
        Calculate t^{k+1}_{EN_i} using (14);
    end for
    Use (15) to obtain SET^{EN};
    select the S nodes with the highest reputation as the consensus node set EN^{con}.
    After completing the calculation task, make an evaluation and update c'_nm;
    Update c'_nm and t^{k+1}_{MUD_n} to blockchain;
end for
return EN^{con}.
```

the EigenTrust-TW reputation management algorithm improves the probability of reaching the normal phase by selecting trusted nodes for consensus participation. Compared to PBFT, this mechanism simplifies the consensus process, reduces inter-node communication and the number of participating nodes, and significantly enhances transaction processing efficiency. These optimizations make RBFT-SMaRt particularly well-suited for latency-sensitive MEC scenarios. Moreover, RBFT-SMaRt retains the modularity of BFT-SMaRt, ensuring adaptability for practical deployment.

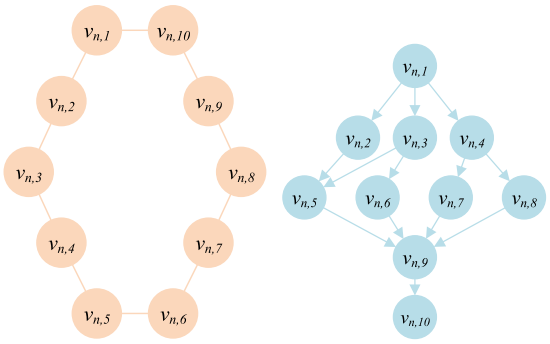
4. Efficient edge-end collaborative computing resource allocation for complex workflows

In this section, we will focus on solving the MaOEECCO model for the offloading problem of computationally intensive tasks involving multiple users and servers. The model accommodates complex workflow applications with both dependencies and independencies. To solve this complex model, we propose a fast converging MaOEA-FT algorithm, which includes adaptive adjustments of crossover and mutation rates based on fitness function metrics, as well as non-dominated sorting.

4.1. Complex workflow task model

In MaOEECCO model, each workflow task has a specific operation. These operations can generally be classified into two types: those with deadlines and strict sequencing requirements, and those without time or order constraints. Accordingly, this article classifies these tasks as dependent workflow and independent workflow, respectively.

Assuming that the task of device MUD_n is an independent workflow, the task is defined as V_n, which is divided into I sub-tasks, where $V_n = \{v_{n,i} | i \in [1, I]\}$, $v_{n,i}$ represent the sub-tasks of the application. The execution process of an independent workflow is shown in Fig. 3(a), and there is no order requirement between sub-tasks. The execution process of dependency workflow is shown in Fig. 3(b). Inspired by [7],



(a) Independent workflow. **(b)** Dependent workflow.

Fig. 3. Two different complex task models.

the application generated by user MUD_n can be modeled as a directed acyclic graph $G_n = (V_n, E_n)$, where $E_n = \{e_{v_{n,i},v_{n,j}} | (i, j) \in [1, I] \times [1, I]\}$, $|E_n| = e$ indicates the dependency relationship between tasks. As shown in Fig. 3(b), task $v_{n,7}$ must start executing after $v_{n,4}$ is completed.

To better represent the computing process of complex workflow task model, we characterize task V_n as a triplet $(C_{n,i}, D_{n,i}, \iota_{n,i})$, where $C_{n,i}$ represents the CPU cycle required to execute task $v_{n,i}$, $D_{n,i}$ represents the data size that task $v_{n,i}$ needs to process, $\iota_{n,i} \in [0, M+1]$ represents the execution location of task $v_{n,i}$, and $\iota_{n,i} = 0$ means that MUD_n executes task $v_{n,i}$ locally. $\iota_{n,i} \in [1, M]$ and $\iota_{n,i} = M+1$ means that task $v_{n,i}$ is offloaded to the edge and the cloud for execution, respectively.

4.2. Objective function

Inspired by [10], this part focuses on the task model for task offloading and details the objective function. Our approach assumes that servers can handle the maximum possible computing tasks within their specified load limits.

4.2.1. Execution time function

The execution time of a task consists of communication time and computation time. Since the amount of data generated after computation is negligible, the return time after computation is not considered in this paper. The communication time for task $v_{n,i}$ can be defined as

$$T^{trans}_{n,i} = \frac{D_{n,i} \cdot d_{n,i}}{S^{up}_{n,i}}, \tag{16}$$

where $d_{n,i}$ represents the unloading distance of task $v_{n,i}$. If the MUD computes task $v_{n,i}$ locally, then $d_{n,i} = 0$. $D_{n,i}$ represents the processing data size of task $v_{n,i}$. $S^{up}_{n,i}$ represents the rate at which task $v_{n,i}$ is offloaded to the server, denoted as

$$S^{up}_{n,i} = W \log_2(1 + \frac{P_{n,i} \cdot H_{n,i}}{\delta^2 + P_{n,i} \cdot \bar{\omega}}), \tag{17}$$

where W represents the communication channel bandwidth of task $v_{n,i}$ in Mbps. $P_{n,i}$ denotes the upload power in watt, $H_{n,i}$ represents the channel gain, δ signifies the channel noise in dBm, and $\bar{\omega}$ denotes the channel interference.

The calculation time of a task is the ratio of task data size to resource execution capability.

$$T^{MUD}_{n,i} = \frac{C_{n,i}}{f_{MUD_n} \cdot \sigma_{MUD_n}}, \tag{18}$$

$$T^{EN}_{n,i} = \frac{C_{n,i}}{f_{EN_m} \cdot \sigma_{EN_m}}, \tag{19}$$

where f_{MUD_n} and f_{EN_m} denote the CPU cycles per second that MUD_n and EN_m can execute in MHz, respectively. σ_{MUD_n} and σ_{EN_m} represent the load status (ideal, normal, overload, full load) of MUD_n and EN_m, respectively.

Therefore, the execution time of the application generated by MUD_n can be defined as follows:

$$T = \sum_{n=1}^{N}\sum_{i=1}^{I} T_{n,i}^{trans} + \sum_{n=1}^{N}\sum_{i=1}^{AP_n} T_{n,i}^{MUD} + \sum_{m=1}^{M}\sum_{i=1}^{AP_m} T_{n,i}^{EN}, \tag{20}$$

where AP_n and AP_m represent applications generated by MUD_n performing calculations in MUD_n and EN_m.

4.2.2. Energy consumption function

In edge computing, the variation in offloading energy consumption between MUDs and ENs affects the overall allocation of computing resources. Additionally, the communication-related energy consumption during offloading cannot be neglected. The energy consumption of offloading can be defined as

$$E = E_{trans} + E_{MUD} + E_{EN}, \tag{21}$$

$$E_{trans} = \sum_{n=1}^{N}\sum_{i=1}^{I} T_{n,i}^{trans} \cdot E_{con}, \tag{22}$$

$$E_{MUD} = \sum_{n=1}^{N}\sum_{i=1}^{AP_n} C_{n,i} \cdot E_{n,i}^{MUD}, \tag{23}$$

$$E_{EN} = \sum_{m=1}^{M}\sum_{i=1}^{AP_m} C_{n,i} \cdot E_{n,i}^{EN}, \tag{24}$$

where E_{con} denotes the transmission power in watt. $E_{n,i}^{EN}$ and $E_{n,i}^{MUD}$ denote the energy consumption to compute task $v_{n,i}$ on the MUD_n and EN_m with load status σ_{MUD_n} and σ_{EN_m} in MJ/GHz.

4.2.3. Load balancing function

When multiple tasks request are offloaded to the same MUD or EN, it slows down computation and increase energy consumption. Not handling such task requests from other MUDs and ENs also wastes computational resources. Therefore, we must consider the load balancing of MUDs and ENs, which can be defined as

$$L = L_{MUD} + L_{EN}, \tag{25}$$

$$L_{MUD} = \sqrt{\frac{\sum_{n=1}^{N}(\sum_{i=1}^{AP_n} D_{n,i} - Limit_n)^2}{N}}, \tag{26}$$

$$L_{EN} = \sqrt{\frac{\sum_{m=1}^{M}(\sum_{i=1}^{AP_m} D_{n,i} - Limit_m)^2}{M}}, \tag{27}$$

where L_{MUD} and L_{EN} represent the loads of MUD and EN in MB, respectively. $Limit_n$ and $Limit_m$ represent the load limits of MUD_n and EN_m in MB, respectively. The standard deviation formula is chosen because the standard deviation between the node task size and the load limit can reflect whether the node is load balanced or not.

4.2.4. Reward function

The owners of mobile devices and edge servers in the MEC system are rational and self interested individuals who care more about their own interests than the performance of the whole system, especially when the edge servers are owned by multiple third-party companies [9]. Therefore, we incorporate the overall rewards of MUDs and ENs into the optimization goal of computing resource allocation. The reward value can be expressed as

$$R = R_{MUD} + R_{EN}, \tag{28}$$

$$R_{MUD} = \sum_{n=1}^{N}\sum_{i=1}^{AP_n} r_{n,i}^{MUD} + \sum_{m=1}^{M}\sum_{i=1}^{AP_m}(r_{n,i}^{MUD} - r_{n,i}^{EN}), \tag{29}$$

$$R_{EN} = \sum_{m=1}^{M}\sum_{i=1}^{AP_m}(r_{n,i}^{EN} - cost_{n,i}^{EN}), \tag{30}$$

where $r_{n,i}^{MUD}$ represents the reward for MUD_n completing task $v_{n,i}$. $r_{n,i}^{EN}$ represents the cost incurred by MUD_n in offloading task $v_{n,i}$ to EN. $cost_{n,i}^{EN}$ is the cost consumed by EN to execute task $v_{n,i}$.

Table 2
Illustration of genetic coding.

	$Sub\text{-}task_1$	$Sub\text{-}task_2$	…	$Sub\text{-}task_I$
MUD_1	χ_{11}^t	χ_{12}^t	…	χ_{1I}^t
MUD_2	χ_{21}^t	χ_{22}^t	…	χ_{2I}^t
…	…	…	…	…
MUD_N	χ_{N1}^t	χ_{N2}^t	…	χ_{NI}^t

4.3. Problem formulation and constraints

The objective of task offloading is to minimize execution time, energy consumption, and load balancing, while maximizing system rewards. Therefore, the optimization objectives of the MaOEECCO task offloading strategy can be described as

$$\min\{T, E, L\}, \tag{31}$$

$$\max\{R\}, \tag{32}$$

constraints:

$$0 \le D_{n,i} \le D_n, \forall n \in [1, N], \tag{33}$$

$$0 \le \sigma_{MUD_n} \le 1, \forall n \in [1, N], \tag{34}$$

$$0 \le \sigma_{EN_m} \le 1, \forall m \in [1, M], \tag{35}$$

$$0 \le \sum_{i=1}^{AP_n} D_{n,i} \le Limit_n, \forall n \in [1, N], \tag{36}$$

$$0 \le \sum_{i=1}^{AP_m} D_{n,i} \le Limit_m, \forall m \in [1, M], \tag{37}$$

$$r^* \le r_{n,i}^{EN}, \tag{38}$$

$$0 \le R_{MUD}, 0 \le R_{EN}, \tag{39}$$

where (33) ensures that the data size of each subtask does not exceed the task size required by each MUD to execution. (34) and (35) enforce that the load status of all MUDs and ENs does not exceed 1. (36) limits the total data size of subtasks executed locally by a device to not exceed its local execution limit. (37) ensures that the total data size of subtasks offloaded to an edge server does not exceed its execution capacity. (38) guarantees that the payment made to servers is not lower than a minimum price r^* for each server. (39) ensures that both MUDs and ENs receive positive rewards.

4.4. Computing resource allocation using MaOEA-FT

The MaOEA-FT algorithm can adaptively adjust the mutation and crossover rates to balance the above four objectives, and the detailed procedure is as follows:

4.4.1. Genetic encoding

In this article, chromosomes are encoded as matrices, a method widely used in various fields [25]. Each chromosome contains N MUDs, with each MUD having I subtasks, resulting in a matrix size of $N \times I$. Inspired by [7], we employ a server sequential nesting strategy to encode the computation offloading problem for complex tasks. Without loss of generality, Table 2 explains the coding of chromosome χ^t in the t_{th} iteration, where rows represent users and columns lists subtasks. χ_{ni}^t is a binary that represents the i_{th} subtask $v_{n,i}$ of the MUD_n in the t_{th} iteration, offloaded to the position represented by ι in the specified order φ.

$$\chi_{ni}^t = (\iota_{ni}^t, \varphi_{ni}^t). \tag{40}$$

The genetic encoding of MUD_n with 10 subtasks is shown in Fig. 4. This encoding corresponds to both workflow patterns depicted in Fig. 3: independent and dependent.

Sub-task	1	2	3	4	5	6	7	8	9	10
Independent Encoding	0, 0	1, 0	4, 0	5, 0	2, 0	6, 0	7, 0	9, 0	8, 0	3, 0
Dependent Encoding	0, 0	3, 1	1, 1	2, 1	1, 2	3, 2	6, 2	4, 2	5, 3	0, 4

Fig. 4. Genetic encoding methods for two workflows.

Algorithm 2 Initialize and Identify Complex Workflows

Input: The subtask $v_{n,i}$, the number of users N, the number of sub-tasks I, the number of ENs M, the size of populations P_S.
Output: The first generation population P^0.

Init empty population P^0;
for $s = 1$ to P_S **do**
  Init empty chromosome χ^0;
  Init empty individual P_s^0;
  for $n = 1$ to N **do**
    for $i = 1$ to I **do**
      $\iota_{ni}^0 = rand\{0, 1, ..., M+1\}$;
      if $e_{v_{n,i},v_{n,j}} \neq 0$ **then**
        Sub-Task $v_{n,i}$ have dependencies;
        Assign values to order φ_{ni}^0 based on dependency relationships $e_{v_{n,i},v_{n,j}}$;
      else
        Sub-Task $v_{n,i}$ have independencies;
        $\varphi_{ni}^0 = 0$;
      end if
      $\chi_{ni}^0 = (\iota_{ni}^0, \varphi_{ni}^0)$;
    end for
  end for
  Calculate $F(\chi^0)$ by (20), (21), (25), and (28);
  $P_s^0 = \chi^0$, $P^0 = P^0 \cup P_s^0$
end for
return P^0;

4.4.2. Fitness function and constraints

The fitness function is an important metric for evaluating computational performance. In an optimization scheme with the objective of minimization, chromosomes with lower fitness values are more likely to persist in subsequent iterations, making this strategy an effective optimization approach.

Our proposed method aims to minimize time and energy consumption while ensuring optimal load balancing, all within strict constraints on data volume and processing load. Simultaneously, we strive to enhance the comprehensive rewards between MUDs and ENs. It is worth noting that any scenario that does not satisfy the pre-defined constraints (33)–(39) will be considered as a failing scenario and thus eliminated in the evolution process. In summary, for this many-objective problem, we have designed a fitness function that comprehensively measures the four core optimization objectives.

$$F(\chi^t) = [T(\chi^t), E(\chi^t), L(\chi^t), R(\chi^t)]. \tag{41}$$

4.4.3. Initialize and recognize complex workflow mode

Based on the coding definition above, the initialization process and the recognition method of complex task patterns are further elaborated. Since task patterns involve two different workflows, identifying the type of workflow is a necessary step. Additionally, the sequence φ must be clearly determined before the initialization phase.

Tasks in a dependency workflow include predecessor and successor tasks, whose order of execution is determined by the dependencies and is reflected in the processing order φ. In contrast, tasks in independent workflows lack direct dependencies, so the processing order φ is uniformly set to zero, indicating no specific processing order is required. The primary task in sequence design is to randomly generate the initial set of chromosomes, effectively avoiding premature convergence. Assuming each offloading strategy corresponds to a chromosome set χ, the specific steps of this initialization process are explained in Algorithm 2.

Algorithm 3 Population Evolution

Input: The initialized population P^0, the size of populations the P_S, the maximum generation Gen
Output: The final population P^{Gen}

for $t = 1$ to Gen **do**
  for $P_s^t \in P^{t-1}$ **do**
    Calculate the fitness function $F(P_s^t)$ according to (41);
  end for
  Calculate σ_F^t and then crossover rate r_c and mutation rate r_m according to (42);
  P^{t-1} conducts mutation and crossover operation to produce S^t;
  Total population Y^t with $2P_S$ population size = $P^{t-1} \cup S^t$;
  Using non-dominated sorting divide Y^t into K fronts ξ_k;
  Init the empty population P^t;
  while $|P^t| < P_S$ **do**
    $P^t = P^t \cup \xi_i$;
    The last front is saved as $\xi_l = \xi_i$;
    $i = i + 1$;
  end while
  if $|P^t| > P_S$ **then**
    Using crowding degree selection for ξ_l;
    According to the niching procedure to select the last candidate chromosomes $\phi = |P^t| - P_S$ from ξ_l;
    Drop ϕ chromosomes in P^t according to crowding degree selection;
  end if
end for
return P^{Gen};

4.4.4. Crossover and mutation

Crossover and mutation operators in evolutionary algorithms generate new solutions that increase population diversity. Typically, smaller mutation and crossover rates lead algorithms to fall into local optimums, while larger rates can lead to excessive computational requirements and convergence difficulties [9]. The constant mutation and crossover rates of NSGA-III [26] ted to limits the evolutionary search process in practical applications, and may produce offspring that are not adapted to the environment. To address this problem, the MaOEA-FT algorithm introduces a fitness tracking strategy to dynamically adjust crossover and mutation rates. The strategy associates these parameters with a fitness function, enabling adaptive optimization of crossover and mutation probabilities to better guide the search process. The crossover rate r_c and mutation rate r_m are denoted as

$$r_c = \frac{\alpha_2 \ln(\sigma_F + 1)}{1 + \alpha_2 \ln(\sigma_F + 1)}, r_m = \frac{\beta_2 \ln(\sigma_F + 1)}{1 + \beta_2 \ln(\sigma_F + 1)}, \tag{42}$$

where $\sigma_F = \sqrt{\frac{\sum_{i=1}^{N}(F_i - \bar{F})^2}{N}}$, $\bar{F} = \frac{1}{N}\sum_{i=1}^{N} F_i$. α_2 and β_2 are parameters that regulate the correlation between mutation rate, crossover rate, and fitness function. Notably, the mutation operation only affects the unloading position ι, while the calculation order φ remains unchanged after initialization.

4.4.5. Population evolution

After performing crossover and mutation operations, new chromosomes are generated. Iteration through Algorithm 3 to produce the

Table 3
Simulation parameters.

Variable	Description	Value
P_S	Population size	100
Gen	Maximum number of iterations	1100
	Objective number	4
	Communication range	2 × 2 km
α_1, β_1	The weights of positive and negative evaluations	0.4, 0.6
α_2, β_2	Influence factors for mutation and crossover rates	3, 0.1
$D_{n,i}$	Data size of task $\upsilon_{n,i}$	100~1300 MB
$C_{n,i}$	CPU cycles required by $\upsilon_{n,i}$	100~1000 MHz
$Limit_n$	Load limitation of UE_n	3 GB
$Limit_m$	Load limitation of EN_m	50 GB
$P_{n,i}$	MUD upload power of task $\upsilon_{n,i}$	100~500 W
δ	Communication noise	(−10)~(−50) dBm
$\bar{\omega}$	Communication interference	0.01~0.02
W	Communication bandwidth	20~40 Mbps
$H_{n,i}$	Communication gain of $\upsilon_{n,i}$	0.2~0.4
f_{MUD_n}	MUD computing power	200~400 MHz
f_{EN_m}	EN computing power	700~900 MHz
E_{con}	Transmission power of MUD	0.3~0.5 W
$E_{n,i}^{MUD}$	MUD calculate the energy consumption per cycle of task $\upsilon_{n,i}$ under load state σ_{MUD_n}	0.7~0.9 MJ/GHz
$E_{n,i}^{EN}$	EN calculate the energy consumption per cycle of task $\upsilon_{n,i}$ under load state σ_{EN_m}	1.1~1.5 MJ/GHz
$r_{n,i}^{MUD}$	Reward of completing task $\upsilon_{n,i}$	3~8 $
$r_{n,i}^{EN}$	Cost of unloading task $\upsilon_{n,i}$	0.3~5 $
$cost_{n,i}^{EN}$	Cost of completing $\upsilon_{n,i}$ using EN	0.2~3 $

final population P^{Gen}, which includes the final computational resource allocation strategy, where Gen is the maximum number of generations.

In comparison to other evolutionary algorithms, the non-dominated sorting method used in NSGA-III exhibits superior performance in convergence for many-objective problems, especially those with three or more objectives. Hence, to solve the MaOEECCO model, we used a non-dominated sorting method similar to NSGA-III to generate a new population.

The non-dominated sorting operation involves selecting chromosomes from the current population P^t based on their fitness function $F(\chi^t)$. Initially, the current population P^t undergoes mutation and crossover to produce the offspring population S^t. Subsequently, the parent population and offspring population are combined into a total population Y^t of size $2P_S$. Finally, based on their fitness values, the total population Y^t is partitioned into K fronts, and each front $k \in K$ is incorporated into an empty set P^t until P^t reaches the population size P_S.

5. Simulation result and security analysis

In this section, we first perform a security analysis of the BICRA framework. Subsequently, we perform extensive simulations to evaluate its performance. Initially, we perform a comprehensive comparison of the proposed RBFT-SMaRt consensus protocol with other consensus protocols. Next, we perform a detailed performance analysis of the EigenTrust-TW algorithm. We then experimentally verify the effectiveness of the MaOEA-FT algorithm for computing resource allocation in MEC system with edge-end collaboration. Finally, we compare the results of BICRA with specific numbers from related frameworks to demonstrate the improvement of BICRA. All simulation experiments were executed on a 64-bit Windows 10 operating system using Java, running on an Intel i7-7700 processor at 3.60 GHz with 16 GB RAM. Inspired by [10,27], the core simulation parameters are listed in Table 3, which describes in detail the specific settings of the research work.

5.1. Malicious data attack

5.1.1. Information spoofing attack

Malicious MUDs may submit fake computational tasks to ENs, leading to unnecessary consumption of computational resources and network congestion. To mitigate such attack, in the proposed blockchain-based sharing architecture, ENs prevent unauthorized MUDs access by verifying certificates. Furthermore, if ENs cannot provide effective services and invade privacy, MUDs can provide the lowest service evaluation, thereby preventing ENs from being selected as the unloading destination by other MUDs [28].

5.1.2. Malicious rating attack

Malicious MUDs can generate unfair evaluations that affect EN's reputation. The BCRC carefully verifies the basis of MUD's evaluations before recording them on the blockchain. If inconsistencies in the MUD's geographical location information is detected, BCRC will reject the upload of unfair reviews. In addition, EigenTrust-TW introduces pre-trusted nodes, which can effectively avoid malicious rating attack [23].

5.1.3. Sybil attack

The Sybil attack involves attackers manipulating or creating multiple virtual ENs to infiltrate and disrupt the system's protective mechanisms [29]. However, these virtual ENs encounter a significant obstacle: their initial trust level is low. Elevating their trust levels to a level of threat requires considerable time. The framework ensures ENs participate in the consensus process once their trust value reaches a threshold, effectively mitigating substantial threats to system security.

5.1.4. On–off attack

The On–Off attack strategy requires the attacker initially employ legitimate behaviors to build trust and establish credibility with other network participants [30]. Subsequently, the attacker shifts to malicious activities, including the dissemination of false information. Notably, if an attacker gains a high level of trust and participates in the consensus mechanism, multiple rounds of voting ensure decentralized decision-making. As a result, it is difficult for a individual attacker to directly influence the consensus outcome, thereby enhancing the system's ability against threats.

5.2. Performance of proposed RBFT-SMaRt

In this part, we compare the performance of PBFT [19], BFT-SMaRt [22] and RBFT-SMaRt consensus protocols using the Java. We configure the number of consensus nodes S to 60 and conduct 200 tests. To improve reliability and generality, we analyze the average results from every 10 tests. Our main criterion for consensus is the accurate output of transaction results, which indicates successful consensus among nodes in the system.

5.2.1. Transaction latency

Transaction latency is the duration from the time a client node sends a transaction request to the master node to the time the client receives a transaction confirmation message. In this experiment transaction latency is measured by inserting timestamps in the log to record the sending time of the transaction request and the confirmation time of the transaction. The test results are presented in Fig. 5, RBFT-SMaRt demonstrates significant advantage over PBFT and BFT-SMaRt in terms of transaction response time. Specifically, when the number of node reaches 100, the average response time of RBFT-SMaRt is 4.9 ms, which is about 45.56% shorter than the 9 ms of PBFT. This advantage becomes even more significant when the number of nodes increases to 200: the average latency of RBFT-SMaRt is 8.3 ms, compare to 27 ms for BFT-SMaRt and 33 ms for PBFT, indicates a maximum latency reduction of approximately 74.85% in RBFT-SMaRt.

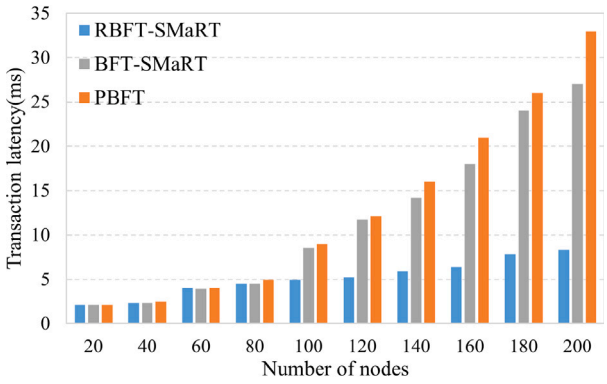


Fig. 5. The transaction latency of the three consensus protocols.

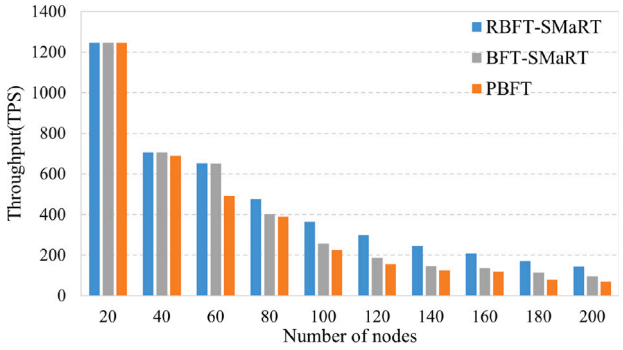


Fig. 6. The throughput of the three consensus protocols.

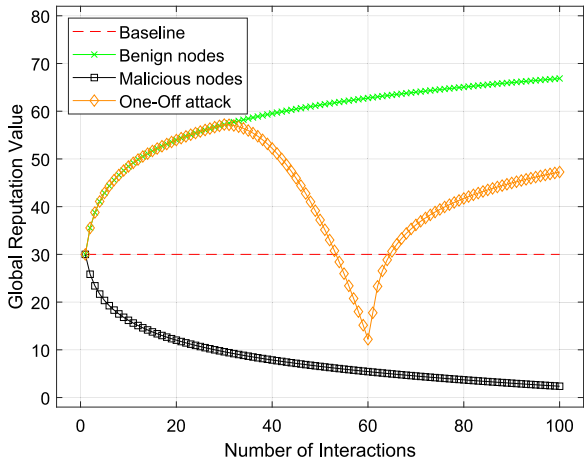


Fig. 7. Trust value changes in different scenarios.

These results are primarily attributed to RBFT-SMaRt's EigenTrust-TW reputation management mechanism, which identifies and selects reliable nodes. This mechanism increases the likelihood of the consensus process entering the normal phase, simplifying operations compared to PBFT and optimizing information exchange among nodes. Moreover, it significantly reduces the number of nodes participating in the consensus process and minimizes the dissemination of information, thereby reducing transaction latency.

5.2.2. Throughput

Throughput quantifies the number of transactions that a system can complete per unit of time, typically expressed in transactions per second (TPS). In our experiment, we configure the client to send 3000 requests and carefully record the number of transactions completed per second for different node configurations to analyze transaction throughput performance. Observing Fig. 6, the advantages of the RBFT-SMaRt algorithm is particularly clear: with 100 nodes, it achieves a high throughput of 363.03 TPS, indicating an increase in maximum throughput of about 60.75%. Even with 200 nodes, the algorithm maintains an efficiency level of 144.60 TPS, highlighting its superiority in throughput performance. RBFT-SMaRt incorporates a node selection mechanism that enhances its network efficiency compared to PBFT and BFT-SMaRt, resulting in improved throughput. This efficiency makes RBFT-SMaRt particularly suitable for the time sensitivity of dynamic MEC scenarios.

5.3. Performance of proposed EigenTrust-TW

In this part, we validate the effectiveness of the proposed reputation evaluation algorithm by simulating three different node behaviors within the BICRA framework using the Matlab 2023b software platform. The experimental setup consist of 100 interactions among nodes, each with a reputation value initialized to 30.

Fig. 7 illustrates the dynamic evolution of reputation values for benign and malicious nodes. The yellow line indicates the fluctuation of trust values under on-off attacks: the attacker initially behaves well for 30 interactions, spreads false information for 30 to 60 interactions, and then resumes good behavior. The results show that our EigenTrust-TW algorithm effectively identifies and resists on-off attack patterns, ensuring robust and reliable reputation evaluation. A consistent observation across all the previously mentioned scenarios is that the reputation growth of malicious ENs is disproportionately small compared to the decline. This observation suggests that malicious ENs need more time to increase their reputation value and therefore can counter Sybil attack. The results suggest that the EigenTrust-TW algorithm can continuously maintain a node's reputation assessment in a dynamic environment. This has the potential to diminish the likelihood of attacks on MEC systems.

5.4. Performance of proposed computational resource allocation scheme

In this part, to demonstrate the prove of the MaOEA-FT algorithm, NSGA-III [26], MOEA/D [31], RVEA [32], and MaOEA-BCD [33] algorithms are compared with it using Matlab2023 and PlatEMO [34] platforms. Similar to [7], we set up the following baseline algorithms to evaluate the performance of our algorithm.

- **ATOL**: All tasks of all MUDs are executed locally.
- **ATOE**: All tasks of all MUDs are executed at EN.
- **ATOC**: All tasks of all MUDs are executed at cloud.
- **Random**: Randomly choose an offloading strategy to execute tasks.
- **Greedy**: Each task is greedily chosen for local execution, EN server, or cloud center based on the weighted sum of four objective values.

5.4.1. Convergence performance

Considering that execution time and energy consumption are more crucial for user experience, the weights for latency and energy consumption are set to 0.35, while the weights for load balancing and rewards are set to 0.15. In this experiment, we configure 3 ENs and 25 randomly distributed MUDs. Each MUD has an independent workflow and a dependent workflow, with each workflow decomposed into 10 subtasks. Fig. 8 illustrates the results of 1100 iterations, using identical parameters across all algorithms, to demonstrate the convergence of MaOEA-FT. Notably, all algorithms effectively converge to the optimal values for the four objective functions within 600 iterations.

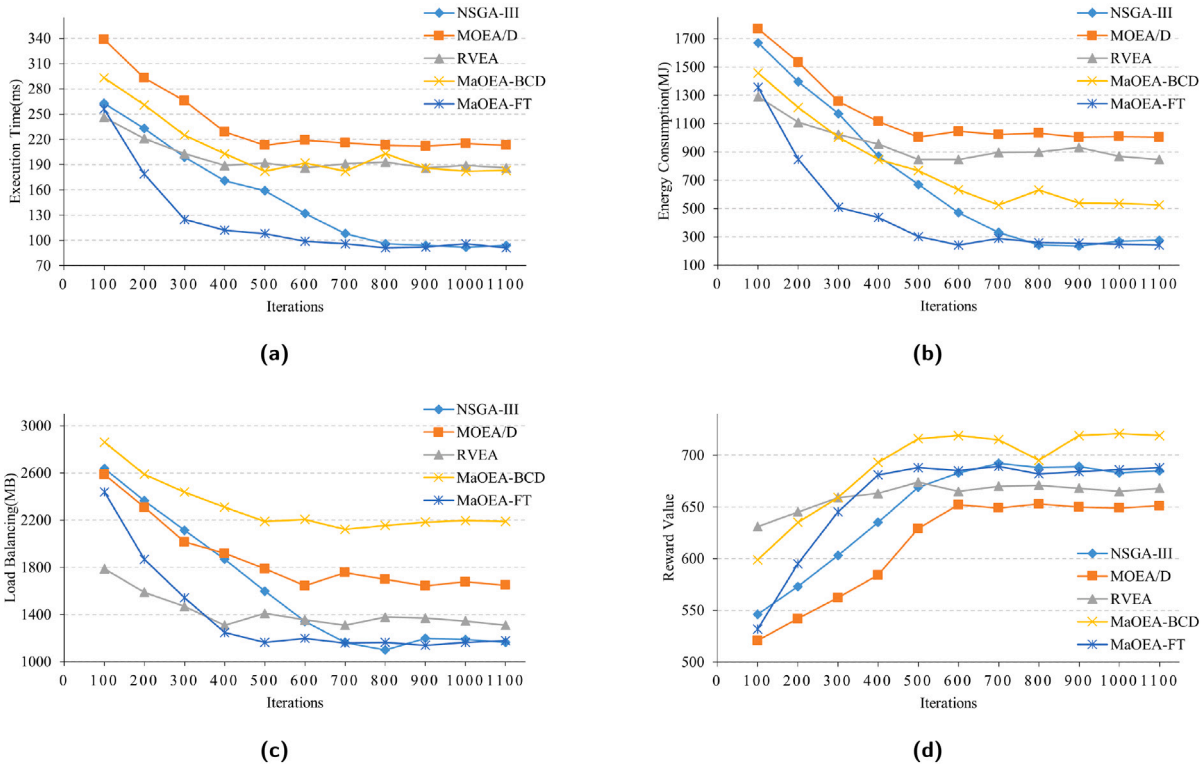


Fig. 8. Evolutionary trends of algorithms in evolution.

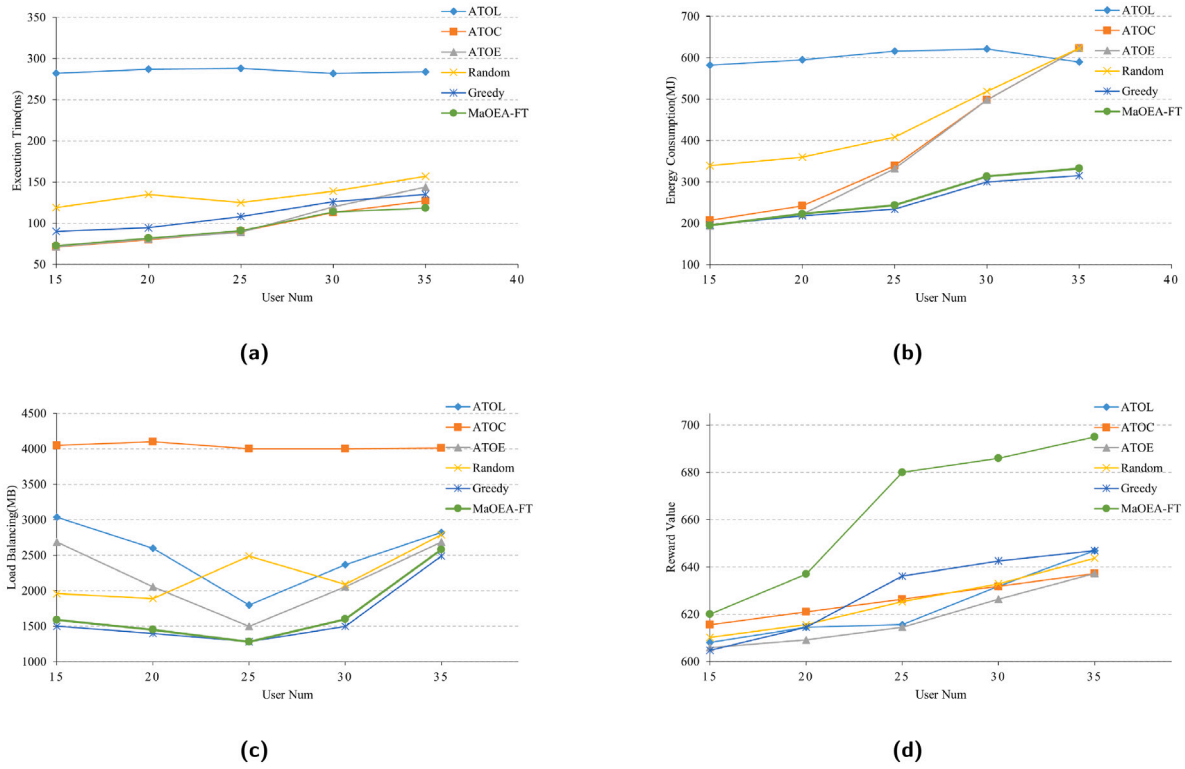


Fig. 9. Execution Time, Energy consumption, Load Balancing, Reward Value with different user numbers.

MaOEA-FT exhibits the fastest convergence in the first 300 iterations, thanks to its ability to adaptively adjust the crossover and mutation rates based on a fitness tracking mechanism. This adaptive mechanism significantly enhances the efficiency of the early search phase and accelerates the initial convergence process of MaOEA-FT by increasing the selection pressure on individuals with low fitness

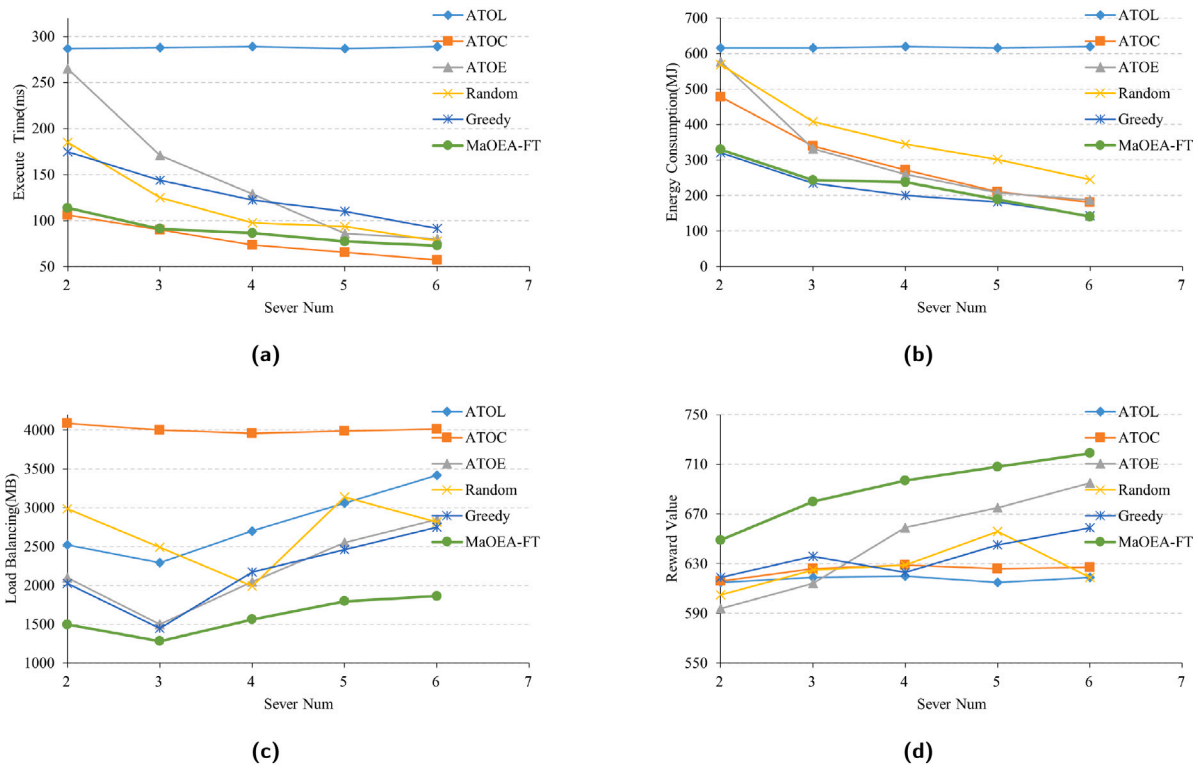


Fig. 10. Execution Time, Energy consumption, Load Balancing, Reward Value with different sever numbers.

values. As the evolutionary process develops, crossover and mutation rates are gradually reduced to prevent premature convergence and to preserve optimal chromosomes, thus appropriately slowing the convergence rate during later iterations (300–500). Compared to other algorithms, MaOEA-FT improves convergence speed by nearly 60%. This improvement is also attributed to the fact that MaOEA-FT encodes the unloading locations and orders into a matrix of decision variables, which integrates strategies with individual characteristics. This matrix greatly enriches the information dimension processed by the algorithm, which in turn enhances solution space exploration, accelerates convergence, and improves global search performance.

5.4.2. Performance of different user numbers

In this experiment, the number of MUDs increases from 15 to 35. There are 3 ENs in the MEC system, and other settings are the same as the section above. As shown in Fig. 9, the MaOEA-FT performs better than the baseline and can effectively balance latency, energy consumption, offloading balance, and reward values. Fig. 9(a) and (b) show that local execution of computing tasks increases latency and energy consumption significantly. Offloading to edge servers and cloud centers reduces latency but raises energy consumption with more users. MaOEA-FT effectively balances these factors. Fig. 9(c) shows the optimal offload balance at around 25 MUDs using three fixed EN servers. Beyond this, EN capacity is exceeded, increasing the offloading equilibrium. Fig. 9(d) shows that the MaOEA-FT algorithm maximizes rewards better than other baselines, confirming its efficacy. Compare to other optimal benchmark algorithms, MaOEA-FT increases the reward value by almost 7.58%, although the other three metrics are very close.

5.4.3. Performance of different EN server numbers

Fig. 10 shows the performance of different algorithms for 25 MUDs with the number of ENs varying from 2 to 6. In Fig. 10(a), ATOL has significantly higher execution times than ATOE, and this gap widens with more ENs. ATOE, benefiting from cloud servers' richer computing resources, achieves lower latency, similar to MaOEA-FT. Fig. 10(b) illustrates that both MaOEA-FT and Greedy algorithms have the lowest energy consumption, with MaOEA-FT performing better in execution time. Fig. 10(c) and (d) show that MaOEA-FT outperforms other benchmarks in load balancing and reward value. Overall, offloading tasks to ENs reduces execution time and energy consumption, improves load balancing, increases reward value, and optimizes all four objectives. Overall, MaOEA-FT achieved a 67.74% reduction in load balancing and a 3.35% increase in reward value while maintaining optimal execution time and energy consumption.

5.4.4. Performance of different workflow numbers

Fig. 11 shows the impact of different workflow quantities on performance. In this MEC system, there are 3 ENs and 25 and MUDs. The horizontal axis represents the increment of independent workflow (IW) and a dependent workflow (DW). The execution time, energy consumption, load balancing and reward values of all the algorithms increase as the computational tasks increase. Fig. 11(a) and (b) show that dependent workflows have a greater impact on execution time and energy consumption compare to independent workflows due to their sequential nature, requires completion of preceding subtasks. Compare to the other benchmarks, the MaOEA-FT algorithm's growth in execution time, energy consumption, and load balancing slows down as the number of tasks increases. Additionally, the MaOEA-FT algorithm's reward value increases rapidly with the number of workflows, achieving optimal results across all four objectives and demonstrating excellent performance.

5.5. Performance of proposed BICRA

This section presents the experimental evaluation of the BICRA scheme. In this experiment, we configure 10 ENs, 100 randomly distributed MUDs, and 6 consensus nodes. Except for these parameters, all other experimental conditions are consistent with those described in the previous section. To systematically evaluate the performance of the BICRA scheme, we include two state-of-the-art comparison schemes:

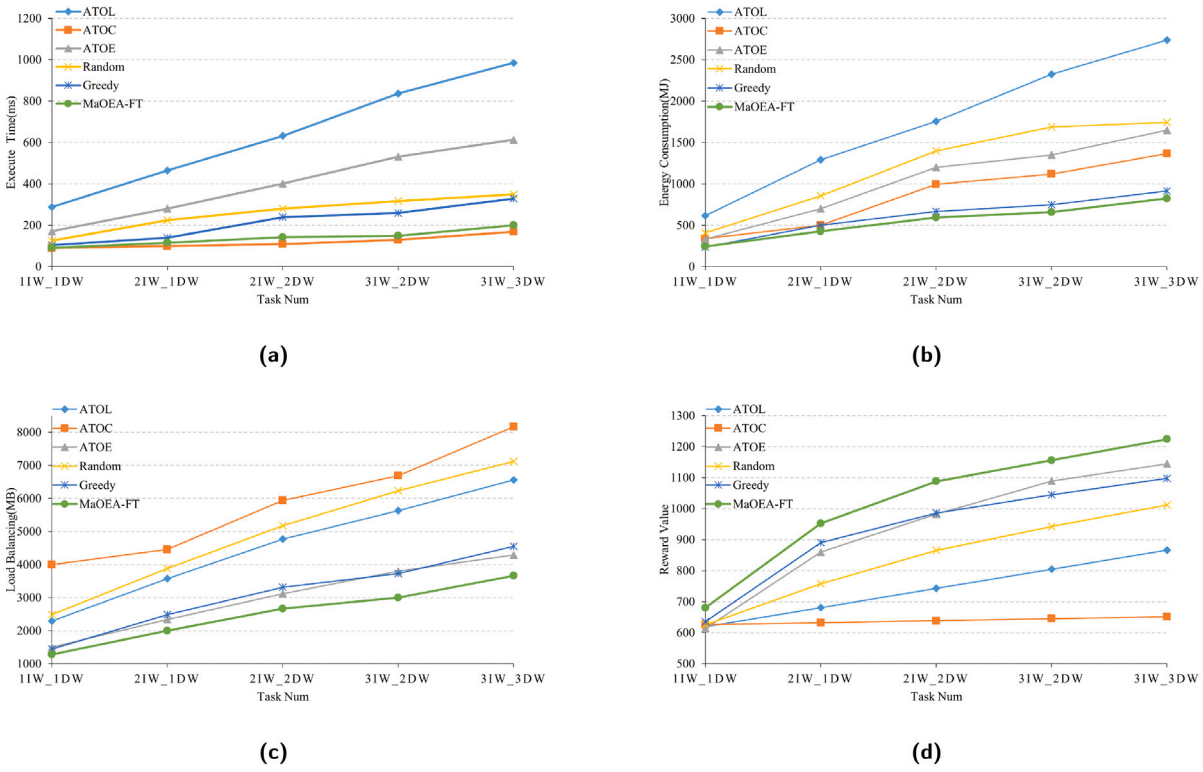


Fig. 11. Execution Time, Energy consumption, Load Balancing, Reward Value with different task numbers.

MOBCA [35] and NRV-MOEA [36]. The evaluation focuses on four key performance metrics: latency, energy consumption, load balancing, and task reward. To ensure fairness and accuracy, 20 independent runs were performed for each algorithm, and the average results are presented in Table 4.

An analysis of the data in Table 4 reveals that BICRA demonstrates superior performance across all four key metrics. Notably, BICRA achieves the lowest energy consumption while maintaining low latency, even with the additional consensus process required for secure task offloading. This performance is attributed to the RBFT-SMaRt mechanism, which reduces transaction processing latency, and the MaOEA-FT algorithm, which accelerates system convergence. The reduced number of iterations enables BICRA to quickly adapt to dynamic environmental changes and efficiently converge to the optimal solution as computational resources fluctuate. Furthermore, the complex workflow task model proposed by BICRA more accurately captures the complexity and dynamic characteristics of real-world problems, enabling it to achieve superior performance across the four key performance metrics. The analysis demonstrates that in real MEC scenarios, BICRA not only enables efficient computing resource allocation but also ensures secure task offloading. Compared to other schemes, BICRA provides higher-quality results, showcasing its exceptional ability to enhance performance while maintaining security.

Table 4
The statistical results of key metrics.

Scheme	Latency (ms)	Energy (MJ)	Load (MB)	Reward
MOEA/D	916.25−	3924.60−	2239.53−	2599.63−
NSGA-III	395.35≈	1078.87−	1580.84+	2735.40≈
RVEA	800.11−	3310.89−	1777.77−	2667.51−
MaOEA-BCD	787.20−	2054.55−	2972.91−	2871.17+
NRV-MOEA	411.63−	1098.29−	1609.30≈	2687.03−
MOBCA	546.28−	1457.56−	2135.72−	2024.72−
BICRA	397.45	949.88	1601.22	2747.38
−/ ≈ /+	5/1/0	6/0/0	4/1/1	4/1/1

−:Comparison scheme is significantly worse than BICRA.
≈:There is no significant difference in the comparative results.
−:Comparison scheme is significantly better than BICRA.

6. Conclusion

We propose BICRA, a blockchain-integrated many-objective optimized edge-end collaborative computational resource allocation framework, to achieve the dual goals of efficient resource allocation and secure task offloading. To address the data leakage problem during task offloading, we propose to use blockchain technology to record the task offloading process to ensure data traceability. To meet the time-sensitive requirements of contemporary MEC systems, we introduce the Eigentrust-TW reputation update model, which supports continuous updating of node reputations in dynamically changing environments, and propose the RBFT-SMaRt fast consensus protocol based on it. In order to better approximate real-world environments, we develop a new complex workflow task model that includes independent and dependent workflows to characterize data-intensive tasks. In addition, we use the MaOEA-FT algorithm to comprehensively consider execution time, energy consumption, load balancing, and task rewards for more efficient computational resource allocation, aiming to adapt to data-intensive tasks in more actual MEC environments. The results of system experiments and comparative evaluations confirm the feasibility and effectiveness of BICRA. In the future, BICRA will be improved based on more user preferences to increase the practicality and efficiency of the proposed solutions.

CRediT authorship contribution statement

Shilong Li: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Conceptualization. **Wenping Ma:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Open Fund of the State Key Laboratory of Intelligent Vehicle Safety Technology under Grant IVSTSKL-202441.

Data availability

No data was used for the research described in the article.

References

[1] H. Verma, N. Chauhan, L.K. Awasthi, A comprehensive review of 'internet of healthcare things': Networking aspects, technologies, services, applications, challenges, and security concerns, Comput. Sci. Rev. 50 (2023) 100591.
[2] K. Sun, J. Wu, Q. Pan, X. Zheng, J. Li, S. Yu, Leveraging digital twin and drl for collaborative context offloading in c-v2x autonomous driving, IEEE Trans. Veh. Technol. 73 (4) (2023) 5020–5035.
[3] S. Machała, N. Chamier-Gliszczyński, T. Królikowski, Application of AR/VR technology in industry 4.0., Procedia Comput. Sci. 207 (2022) 2990–2998.
[4] Y. Mao, C. You, J. Zhang, K. Huang, K.B. Letaief, A survey on mobile edge computing: The communication perspective, IEEE Commun. Surv. Tutor. 19 (4) (2017) 2322–2358.
[5] J. Ren, D. Zhang, S. He, Y. Zhang, T. Li, A survey on end-edge-cloud orchestrated network computing paradigms: Transparent computing, mobile edge computing, fog computing, and cloudlet, ACM Comput. Surv. 52 (6) (2019) 1–36.
[6] M.Y. Akhlaqi, Z.B.M. Hanapi, Task offloading paradigm in mobile edge computing-current issues, adopted approaches, and future directions, J. Netw. Comput. Appl. 212 (2023) 103568.
[7] J. Chen, Y. Yang, C. Wang, H. Zhang, C. Qiu, X. Wang, Multitask offloading strategy optimization based on directed acyclic graphs for edge computing, IEEE Internet Things J. 9 (12) (2021) 9367–9378.
[8] H. Li, H. Xu, C. Zhou, X. Lü, Z. Han, Joint optimization strategy of computation offloading and resource allocation in multi-access edge computing environment, IEEE Trans. Veh. Technol. 69 (9) (2020) 10214–10226.
[9] Y. Gong, K. Bian, F. Hao, Y. Sun, Y. Wu, Dependent tasks offloading in mobile edge computing: a multi-objective evolutionary optimization strategy, Future Gener. Comput. Syst. 148 (2023) 314–325.
[10] Z. Cui, Z. Xue, T. Fan, X. Cai, W. Zhang, A many-objective evolutionary algorithm based on constraints for collaborative computation offloading, Swarm Evol. Comput. 77 (2023) 101244.
[11] K. Peng, H. Huang, B. Zhao, A. Jolfaei, X. Xu, M. Bilal, Intelligent computation offloading and resource allocation in iIoT with end-edge-cloud computing using NSGA-III, IEEE Trans. Netw. Sci. Eng. 10 (5) (2022) 3032–3046.
[12] Z. Cui, Z. Xue, Y. Ma, X. Cai, J. Chen, A many-objective optimized sharding scheme for blockchain performance improvement in end-edge enabled internet of things, IEEE Internet Things J. 10 (24) (2023) 21443–21456.
[13] X. Ye, M. Li, P. Si, R. Yang, Z. Wang, Y. Zhang, Collaborative and intelligent resource optimization for computing and caching in IoV with blockchain and MEC using A3C approach, IEEE Trans. Veh. Technol. 72 (2) (2022) 1449–1463.
[14] M. Masuduzzaman, A. Islam, K. Sadia, S.Y. Shin, UAV-based MEC-assisted automated traffic management scheme using blockchain, Future Gener. Comput. Syst. 134 (2022) 256–270.
[15] S. Guo, X. Hu, S. Guo, X. Qiu, F. Qi, Blockchain meets edge computing: A distributed and trusted authentication system, IEEE Trans. Ind. Inf. 16 (3) (2019) 1972–1983.
[16] A. Samy, I.A. Elgendy, H. Yu, W. Zhang, H. Zhang, Secure task offloading in blockchain-enabled mobile edge computing with deep reinforcement learning, IEEE Trans. Netw. Serv. 19 (4) (2022) 4872–4887.
[17] Q. Wang, S. Chen, M. Wu, Incentive-aware blockchain-assisted intelligent edge caching and computation offloading for IoT, Eng. 31 (2023) 127–138.
[18] R. Zhang, R. Xue, L. Liu, Security and privacy on blockchain, ACM Comput. Surv. 52 (3) (2019) 1–34.
[19] M. Castro, B. Liskov, et al., Practical byzantine fault tolerance, in: OSDI, Vol. 99, 1999, pp. 173–186.
[20] F. Tschorsch, B. Scheuermann, Bitcoin and beyond: A technical survey on decentralized digital currencies, IEEE Commun. Surv. Tutor. 18 (3) (2016) 2084–2123.
[21] S. Nakamoto., Bitcoin: A peer-to-peer electronic cash system., 2008, http://bitcoin.org/bitcoin.pdf.
[22] A. Bessani, J. Sousa, E.E. Alchieri, State machine replication for the masses with BFT-SMART, in: 2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, IEEE, 2014, pp. 355–362.
[23] S.D. Kamvar, M.T. Schlosser, H. Garcia-Molina, The eigentrust algorithm for reputation management in p2p networks, in: Proceedings of the 12th International Conference on World Wide Web, Proc. WWW, 2003, pp. 640–651.
[24] L. Lin, J. Wu, Z. Zhou, J. Zhao, P. Li, J. Xiong, Computing power networking meets blockchain: A reputation-enhanced trading framework for decentralized IoT cloud services, IEEE Internet Things J. 11 (10) (2024) 17082–17096.
[25] Z. Hua, K. Zhang, Y. Li, Y. Zhou, Visually secure image encryption using adaptive-thresholding sparsification and parallel compressive sensing, Signal Process. 183 (2021) 107998.
[26] K. Deb, H. Jain, An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints, IEEE Trans. Evol. Comput. 18 (4) (2013) 577–601.
[27] J. Kang, R. Yu, X. Huang, M. Wu, S. Maharjan, S. Xie, Y. Zhang, Blockchain for secure and efficient data sharing in vehicular edge computing and networks, IEEE Internet Things J. 6 (3) (2018) 4660–4670.
[28] Q.-u.-A. Arshad, W.Z. Khan, F. Azam, M.K. Khan, H. Yu, Y.B. Zikria, Blockchain-based decentralized trust management in IoT: systems, requirements and challenges, Complex Intell. Syst. 9 (6) (2023) 6155–6176.
[29] Y. Tang, J. Yan, C. Chakraborty, Y. Sun, Hedera: a permissionless and scalable hybrid blockchain consensus algorithm in multi-access edge computing for iot, IEEE Internet Things J. 10 (24) (2023) 21187–21202.
[30] W. Liu, B. Cao, M. Peng, Blockchain based offloading strategy: Incentive, effectiveness and security, IEEE J. Sel. Areas. Commun. 40 (12) (2022) 3533–3546.
[31] Q. Zhang, H. Li, MOEA/D: a multiobjective evolutionary algorithm based on decomposition, IEEE Trans. Evol. Comput. 11 (6) (2007) 712–731.
[32] R. Cheng, Y. Jin, M. Olhofer, B. Sendhoff, A reference vector guided evolutionary algorithm for many-objective optimization, IEEE Trans. Evol. Comput. 20 (5) (2016) 773–791.
[33] Z. Zhang, Y. Cao, Z. Cui, W. Zhang, J. Chen, A many-objective optimization based intelligent intrusion detection algorithm for enhancing security of vehicular networks in 6G, IEEE Trans. Veh. Technol. 70 (6) (2021) 5234–5243.
[34] Y. Tian, R. Cheng, X. Zhang, Y. Jin, PlatEMO: A MATLAB platform for evolutionary multi-objective optimization, IEEE Comput. Intell. Mag. 12 (4) (2017) 73–87.
[35] J. Jiang, J. Wu, J. Luo, X. Yang, Z. Huang, MOBCA: Multi-objective besiege and conquer algorithm, Biomimetics 9 (6) (2024) 316.
[36] Y. Hua, Q. Liu, K. Hao, Adaptive normal vector guided evolutionary multi-and many-objective optimization, Complex Intell. Syst. (2024) 1–18.



Shilong Li is currently pursuing the Ph.D. degree with the School of Telecommunications Engineering, Xidian University, Xi'an, China, in 2023. His research interests include, mobile edge computing, computing offloading, reputation management, and blockchain.



Wenping Ma received the B.S. and M.S. degrees in fundamental mathematics from Shaanxi Normal University, Xi'an, China, in 1987 and 1990, respectively, and the Ph.D. degree in communication and information system from Xidian University, Xi'an, in 1999, where he is currently a Full Professor with the School of Telecommunications Engineering. His current research interests include information theory, communication theory, blockchain, and information security.